

Étterem

Programozói

dokumentáció

Antal Csongor

J69X80

Étterem programjának programozói dokumentációja

Az étterem programot egy menürendszerrel irányíthatjuk, amely 8 pontból áll. A program fő változója egy szám lesz (int gomb;) amely bekérésével eljuthatunk a menü egyes opcióinak működéséhez.

A program több függvényből áll össze amiket a főfüggvényen kívül készítettünk el. Ezek a függvények kerülnek meghívásra a főfüggvényben miután megkaptuk, hogy mennyi a gomb értéke. A főfüggvényben a fő ciklus az egy do...while ciklus, amely az egész program működését biztosítja. Ez a ciklus önmagában tartalmaz hét darab logikai döntést (if) amik eldöntik, mit akar a felhasználó, és azt hajtják végre. A programban minden rossz bemeneti adatra értesítést kap a felhasználó.

A program főfüggvényének elején deklarálunk két dinamikus tömböt, ezek lesznek a program két nagyon fontos tömbje. Ezek nem egyszerű tömbök, hanem struktúra tömbök. Ezek deklarálása után egyből meg is hívunk két függvényt, amelyek lefutásával értékadás történik. A rendelés tömböt a rendelés felvétel illetve a számla nyomtatásnál használjuk.

```
menu *rendeletomb = (menu*)malloc(asztalszam*sizeof(menu));
```

```
menu *menutomb = (menu*) malloc(menu_meret * sizeof(menu));
```

```
beolvas_menu(menutomb,menu_meret);
```

A következő függvénnyel beolvassuk láncolt listába az asztalokat:

```
beolvas_asztalok(&asztalokfej, asztalszam);
```

A programban használunk nem alap kellékeket is, ezért szükséges fejláncokat (headers) becsatolni:

```
#include <stdlib.h>
```

```
#include <conio.h>
```

```
#include <stdbool.h>
```

```
#include "debugmalloc.h"
```

A program helyenként vár amíg egy gomb nem kerül megnyomásra. Ezt egy felszólítással is jelzi, hogy nyomjunk ENTER-t. Ehhez a **getch();** illetve a **getchar();** függvényeket használja, ezután a **system("cls");-t** használva újra csak a vezérlő menürendszert látjuk, és választhatunk újra az opciók közül.

A program elején deklarálom a program főelemeit: ez egy struktúra lesz, illetve egy láncolt lista. Az egyikben az asztalok a másikban a menü és rendelés adatai vannak tárolva.

///egy struktúrát hozok létre, amely a menü egyes adatait olvassa be és tárolja

```
typedef struct menu
```

```
{
```

```

int sorszam; ///< ez egy számot kap, hogy hányadik a menüben az étel
char etel[50]; ///< ez az ételt nevét tárolja
int ar; ///< ez az árát tárolja
int rendeles[200];
int voltrendeles;
}menu;

```

///ezt a láncolt listát az asztalok foglaltságának tárolására hozom létre

typedef struct asztal

```

{
    int sorszam; ///< minden asztal egy számot kap és ez ezt tárolja
    int ferohely; ///< ez az asztal férőhelyét tárolja
    int foglalt; ///< ez tárolja, hogy foglalt vagy sem
    struct asztal *kov; ///< következő asztalra mutató pointer
}asztal;

```

A függvények paraméterként az egyik tömböt, vagy láncolt listát, illetve változót kapják be, vagy akár többet is, akár semmit, annak függvényében, hogy mit csinál az adott függvény, mivel dolgozik. A függvényekben *.txt fájlokból olvasunk be.

int beolvas_menu_meret();

A menü méretét olvassa be.

void beolvas_menu(menu *menutomb,int menu_meret);

Ez a függvény magát a menüt olvassa be.

void beolvas_asztalok(asztal **asztalokfej, int asztalszam);

Az asztalok adatait olvassa be, amelyben szerepel az asztal száma, férőhelye, illetve az, hogy foglalt vagy sem.

void asztalfoglalas(asztal *asztalokfej, int asztalszam);

Ez a függvény az asztalok lista foglalt részét változtatja, ezáltal történik foglalás.

void asztalkilistazas(asztal *asztalokfej,int asztalszam);

Kilistázza az asztalainkat és a fent felsorolt adataikat.

void matrmegj();

Ez egy mátrixot jelenít meg fájlból olvasva, amely az étterem asztalainak elhelyezkedését szeretné ábrázolni.

void menumegj();

A menüt írja képernyőre.

void asztal_felszabaditas(asztal *asztalokfej);

Majdnem ugyan az mint az asztal foglalásnál. A kiválasztott asztal foglaltsági adatát módosítja.

void rendelesfelvetel(menu *rendelestomb, int asztalszam, asztal *asztalokfej);

Rendelést vesz fel egy struktúra tömbbe, innen tudjuk melyik asztalhoz tartozik a rendelésünk. Ezt csak akkor tehetjük meg, ha az asztalunk foglalt, ezért van szükségünk az asztal lista paraméter megadására is.

void szamlanyomtatás(menu *rendelestomb, int asztalszam, asztal *asztalokfej, menu *menutomb);

Számlát nyomtat egy adott asztal rendeléséről. Számlanyomtatáskor a bekért asztalt automatikusan felszabadítja, illetve a rendelés törlődik ezáltal számlanyomtatás után egyből lehet újra foglalni, illetve ezután rendelést felvenni.

void asztal_free(asztal *asztalokfej);

Ez a függvény a végén felszabadítja a listát, hogy ne legyen memóriaszivárgás a programban.

free(menutomb);

free(rendelestomb);

A végén a függvény mellett a tömböket szintén felszabadítjuk.

A programban 1-től 7-ig választhatunk opciók közül amik választása után a program végrehajtja azt amit ír, a nyolcas gombra a program bezárul.